Refactoring of LCMsim: A Lightweight Julia Package for Mould Filling Simulations in Liquid Composite Moulding

Christof Obertscheider[1], Leonard Heber[2], Carola Lenzen[2], and Ewald Fauster[3(✉)]

[1] Aerospace Engineering Department, University of Applied Sciences Wiener Neustadt, Wiener Neustadt, Austria
christof.obertscheider@fhwn.ac.at

[2] Institute for Software and Systems Engineering, University of Augsburg, Augsburg, Germany

[3] Processing of Composites Group, Department Polymer Engineering and Science, Montanuniversität Leoben, Leoben, Austria
ewald.fauster@unileoben.ac.at

Abstract. The Julia package LCMsim was developed to verify if the equations for compressible fluid flow through a thin curved cavity can be solved on a shell mesh. Although directly accessible as open source, the code was hard to understand, maintain and extend for outsiders. In this paper the general design of LCMsim after a refactoring step is discussed and general design principles for a numerical tool solving the equations of hydrodynamics are presented and implemented.

LCMsim is designed to run mold filling simulations for Liquid Composite Molding, a manufacturing process for fiber reinforced polymer composites where dry fibers are placed into a mold which is subsequently closed or sealed with a flexible bag by drawing a vacuum. Afterwards, the pressure difference between the injection gates and the vents pushes the resin through the porous preform.

The governing equations are specified as Initial-Boundary-Value problem with partial differential equations including temporal and spatial derivatives. Consequently, this paper can serve as a guidance for scientific computing applications.

The results section quantifies the decrease in computational time for a benchmark problem after refactoring (17 times faster). In addition, a methodology is addressed for modifying existing equations as well as for adding supplementary equations of phenomenological models.

Keywords: Julia · Software architecture · Computational Fluid Dynamics (CFD) · Shell mesh · Liquid composite moulding (LCM) · Filling simulation

G. Ernst et al. (Eds.): Wolfgang Reif Festschrift, LNCS 15765, pp. 132–154, 2025.
https://doi.org/10.1007/978-3-031-92196-4_7

1 Introduction

Resin Transfer Molding (RTM) and Vacuum-Assisted Resin Infusion (VARI) are two examples for Liquid Composite Molding (LCM) technologies [9,19]. In RTM, dry fiber preforms are placed into a matched mold and resin is then injected under pressure. In VARI, dry fiber preforms are placed into a single-sided mold which is subsequently sealed with a flexible bag by drawing vacuum. The vacuum also pulls the resin through the fiber preform.

During mold design, filling simulations can study different manufacturing concepts (i.e. placement of injection gates and vents) to guarantee complete filling of the part and avoid air entrapment where flow fronts converge. Incomplete filling and voids result in a decrease of the mechanical properties (stiffness and strength) of the manufactured part.

An additional degree of freedom for the injection strategy comes into play if multiple injection gates in RTM or multiple vents in VARI are present. Controlling the injection pressure at the individual inlets or controlling the pressure in the individual vents can influence the flow front propagation in order to ensure a uniform, complete and fast impregnation. A more regular spreading of the resin can be achieved by data-driven controller designs using various machine learning models. Training of the machine learning models in a real-world setup is not feasible. A large number of time-consuming and costly experiments was required. Therefore, a numerical simulation of the resin flow is used to provide a simulative environment resembling the filling process.

A software tool which can be used as a simulative environment must be validated with experiments. This is true for PAM-RTM [6] and RTMsim [11] which were already used for this purpose [17,18]. The Julia module RTMsim and its successor LCMsim [12] are both open source and at any point in simulated time allow for changing the injection pressure and obtaining the state of the simulation. This ability is crucial for online control and needs to be possible without terminating the executing process to achieve high efficiency and stability. Multiple parallel instances of the simulation must run on a multi-node computer cluster. Therefore, efficiency is another requirement for such a simulation environment.

The main goal of this paper is to show how a software tool for scientific computing (if the model is defined in terms of an initial-boundary-value (IBV) problem) can be designed in a Julia-like programming language. The main requirements are computational efficiency, easy expandability of the open source code and possible integration into other projects such as a graphical use interface (GUI) or machine learning infrastructure. All requirements for the refactoring of LCMsim and how they are implemented are shown in Sect. 2.2.

The numerical solution of the IBV problem describing the flow front propagation mainly follows standard procedures from Computational Fluid Dynamics (CFD) [7,20]. Section 2.1 presents the IBV problem and discusses special aspects, for example how the mesh handling for the Finite Area Method is different than for the Finite Volume Method.

Finally, Sect. 3 presents quantitative results and how the requirements are fulfilled.

2 Materials and Methods

2.1 Numerical Solution of the IBV Problem

Processes in continuum mechanics are described by partial differential equations with initial and boundary conditions [16]. Methods from continuum mechanics are used to describe processes in solid mechanics, fluid mechanics and heat transfer.

The filling of the cavity is modeled as compressible, two-phase (incompressible oil/resin[1] and compressible air), viscous, laminar, iso-thermal fluid flow through porous media.

From a mathematical point of view resin flow through a porous cavity is described by the following IBV problem. All physical quantities (fluid mixture mass density ρ, filling fraction c, superficial velocity $\boldsymbol{u}$ which describes the resin flow through the porous cavity, pressure p) and all parameters (porosity ε, permeability $\boldsymbol{K}$, dynamic viscosity μ) are functions of space $\boldsymbol{x}$ and time t where $\boldsymbol{x} \in \Omega \subset \mathbb{R}^3$ and Ω is the cavity. Since the flow through the porous cavity is described by the superficial velocity with zero component in thickness direction, the assumption of plug flow is valid and the cavity is described by its mid-surface with cavity thickness as a local property of the geometric model [15].

The governing equations for the default filling model in LCMsim are [15]:

$$\frac{\partial \varepsilon \rho}{\partial t} + \nabla \cdot (\rho \boldsymbol{u}) = 0 \tag{1}$$

$$\frac{\partial \rho \boldsymbol{u}}{\partial t} + \nabla \cdot (\rho \boldsymbol{u}\boldsymbol{u}) = -\nabla p + S_u \tag{2}$$

with the two-component velocity vector $\boldsymbol{u} = (u, v)$ where u and v are aligned with the cell reference directions (see the mesh handling sections for details) and with the fluid mixture mass density $\rho = \gamma\rho_{\text{resin}} + (1 - \gamma)\rho_{\text{air}}$. ρ_{resin} is the resin mass density at injection pressure and ρ_{air} is the air mass density at initial cavity pressure. The binary filling fraction γ is equal to $\gamma = 0$ (i.e. empty) if $\rho < \bar{\rho}$ and $\gamma = 1$ (i.e. completely filled) if $\rho \geq \bar{\rho}$, and the threshold value is $\bar{\rho} = (\rho_{\text{resin}} + \rho_{\text{air}})/2$. S_u is a source term which contains the pressure loss from flow through a porous medium according to Darcy's law, i.e. $S_u = -\mu \boldsymbol{K}^{-1}\boldsymbol{u}$ with dynamic viscosity μ and permeability tensor $\boldsymbol{K}$. The equation of state (EOS) models the pressure rise as the fluid mixture mass density increases. It is assumed that the pressure build up is slow at low levels of fluid mixture mass density (i.e. zero slope $\partial p/\partial \rho$). Depending on the chosen EOS model the slope at ρ_{resin} is changed (smaller slope allows for larger time steps and a very large slope can model the real compressibility of the resin). Figure 1 shows curves for two different EOS.

[1] Oil and resin are used synonymously here. Part manufacturing is performed with resin whereas characterization tests often are performed with oil as a test fluid.

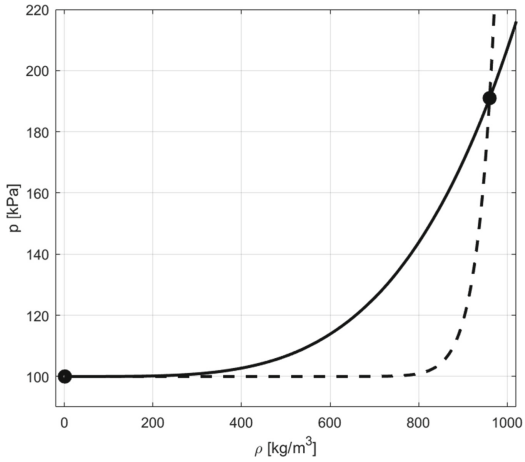


Fig. 1. The equation of state models the pressure build-up for the air-resin mixture. A polynomial fit $c_1 + c_2\rho^\alpha$ through the two mass density/pressure pairs for air at initial cavity pressure and resin at injection pressure is used. The solid curve is for air density $1.225\,\mathrm{kg/m^3}$ at 100000 Pa and resin density $960\,\mathrm{kg/m^3}$ at 191000 Pa which is used in the validation cases from [15]. The dashed EOS curve assumes real compressiblity. The solid EOS curve allows for larger time steps and some numerical diffusion.

Predominant numerical solution techniques for the IBV problem formulated by Eqs. (1) and (2) are the Finite-Element-Method (FEM), the Finite-Difference-Method (FDM) and the Finite-Volume-Method (FVM). The latter is used here. For all methods, the domain is divided into a set of discrete volumes V_i, called cells. This process is called meshing and is shown in Fig. 2. The cells (subfigure b) fill the whole domain Ω (subfigure a) without overlap. Since plug flow describes the filling of the thin-walled porous cavity, only one cell through the thickness is required (subfigure c). Thus, it is more convenient to extract a mid-surface model (subfigure d) first and then spilt this into cells to obtain a shell mesh (subfigure e) instead of a solid mesh.

The fluid flow at a certain time instance is described by values of the physical quantities in every cell. In order to calculate how these values change with the FVM, the fluid-flow equations are volume-integrated over each individual cell. If shell cells (with cavity thickness as cell property) are used instead of solid cells, the method is called Finite-Area-Method (FAM). All spatial derivatives are approximated by finite differences (algebraic relations between the physical quantities in the considered and neighbouring cells) and a set of ordinary differential equations including temporal derivative must be solved. Explicit methods calculate the values for the physical quantities at a later time instance from physical quantities at the current time. Implicit methods use physical quantities from the current and the later state.

The time step Δt in an explicit scheme is limited by the Courant-Friedrichs-Lewy (CFL) condition introduced in [4]. Implicit schemes allow for larger time

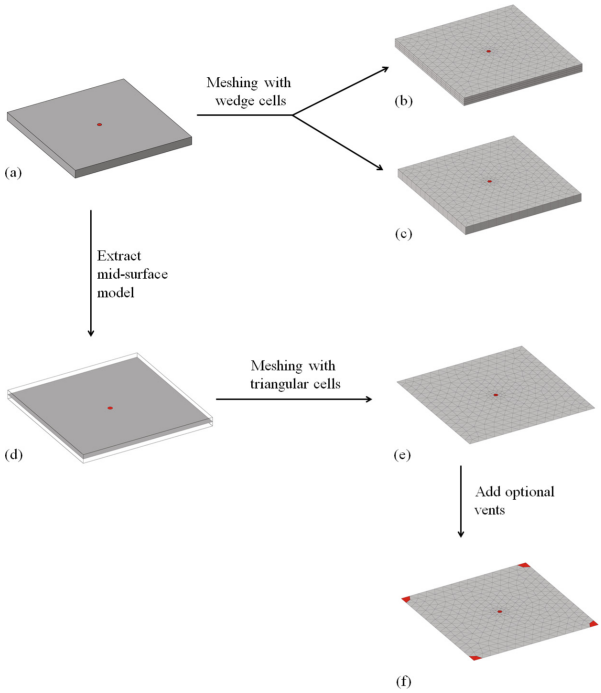


Fig. 2. Top: Solid domain with central inlet. Two different meshes (only one and four cells through the thickness) are shown. Bottom: Mid-surface domain with central inlet. A shell mesh is created. Furthermore, a setup with central inlet and optional vents is shown.

steps Δt at higher computational cost because a large linear system must be solved. An explicit scheme was chosen because it is much easier to add and modify partial differential equations.

A numerical scheme is the result of the FAM discretization. It describes how the values of a physical quantity in a considered cell i after a time increment Δt (indicated by time index $n+1$) can be calculated from values of physical quantities in the considered and neighbouring cells. One can easily see similarities to the IBV problem in the implemented numerical scheme for cell V_i:

$$\rho_i^{n+1} = \left(\varepsilon_i \rho_i^n - \frac{\Delta t}{V_i} \sum_j \boldsymbol{n}_{i,j} \cdot \underbrace{\frac{1}{2}\left(\rho_i^n + \rho_j^n\right) \frac{1}{2}\left(\boldsymbol{u}_i^n + \boldsymbol{u}_i^n\right)}_{(\rho \boldsymbol{u})_{i,j}^n} A_{i,j} \right) / \varepsilon_i \tag{3}$$

$$u_i^{n+1} = \left(\rho_i^n u_i^n - \frac{\Delta t}{V_i} \sum_j \boldsymbol{n}_{i,j} \cdot (\rho \boldsymbol{u})_{i,j}^n u_{i,j}^n A_{i,j} - \left(\frac{\partial p}{\partial x}\right)_i \Delta t \right) / \left(\rho_i^{n+1} + (\mu_i / k_1)\, \Delta t \right) \tag{4}$$

$$v_i^{n+1} = \left(\rho_i^n v_i^n - \frac{\Delta t}{V_i} \sum_j \boldsymbol{n}_{i,j} \cdot (\rho \boldsymbol{u})_{i,j}^n v_{i,j}^n A_{i,j} - \left(\frac{\partial p}{\partial y}\right)_i \Delta t \right) / \left(\rho_i^{n+1} + (\mu_i / k_2)\, \Delta t \right) \tag{5}$$

where i is the index of the considered cell and $j \in S_i$ represents the index of a neighbouring cell and the combined index i, j represents the index of the cell interface between cell i and cell j. Only the Darcy source term is treated implicitly. Otherwise a much smaller time step than given by the CFL condition would be required. The iteration remains explicit because ρ_i^{n+1} is evaluated before u_i^{n+1} and v_i^{n+1}.

The physical quantities $u_{i,j}^n$ and $v_{i,j}^n$ at the cell interface are evaluated using first-order upwinding. For flow from cell i to cell j, the velocity components evaluated using first-order upwinding are $u_{i,j}^n = u_i^n$ and $v_{i,j}^n = v_i^n$. The j-sum runs over all neighbouring cells. When the neighbouring cell is a pressure inlet cell, the velocity in the factor without upwinding in the convective term is evaluated according to Darcy's law.

The solver section for an IBV problem consists of a `for`-loop over all interior and wall cells inside a `while`-loop for time evolution. After every time step, the time step size can be adjusted.

For every interior and wall cell, the following steps must be performed:

(1) The pressure and filling fraction gradients are evaluated.
(2) The time evolution for new mass density, x- and y-velocities with the discretized continuum, x- and y-momentum equations in the cell coordinate systems is performed.
(3) The pressure is evaluated according to the equation of state.
(4) The filling fraction is evaluated.

If boundary conditions in the pressure inlet and outlet cells are changed, a new `while`-loop has to be started.

All details of the described procedure are shown in Algorithm 1.

Parameters. The IBV problem is characterized by a set of parameters. The parameters can be split into process parameters (for example injection pressure p_a and initial cavity pressure p_{init}, mass density of air ρ_{air} at initial cavity pressure and mass density of resin ρ_{resin} at injection pressure, dynamic viscosity μ of the resin) and preform parameters for every cell group (for example: cavity thickness t at initial cavity pressure, permeability k_1 in the first reference direction, ratio k_2/k_1 of in-plane permeability for calculating the permeability k_2 in the second reference direction, porosity ε, directional vector which is projected on to cell to get the reference directions). All input parameters are summarized in Sect. 2.2.

Mesh Handling. As described in Sect. 2.1 and shown in Fig. 2e, the spatial domain is described by a mid-surface model which is meshed with triangles. The cell thickness is assigned as a cell property. A triangular cell represents a wedge cell where flow through the three lateral control surfaces is possible.

The mid-surface model can be curved and cells can have edges where more than two cells are connected to each other such as for handling T-junctions. This is illustrated in Fig. 3a.

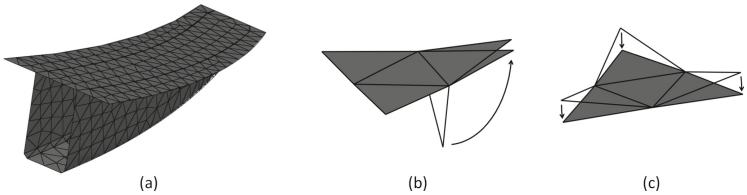


Fig. 3. Annulus filler [2] with T-sections and curved regions (a). Two examples for a locally flat mesh in (b) and (c).

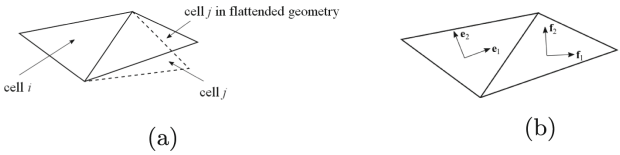


Fig. 4. Cells i and j after flattening (a) and local coordinate systems in cells i and j (b).

For discretizing the equations on the shell mesh of the part's mid-surface it is assumed that the geometry was locally flat there. The neighbouring cells are rotated about the common edges to lie in the plane of the considered cell. This is illustrated in Fig. 3b and 3c respectively. This local flattening of the geometry is an approximation. Pressure losses due to the change in flow direction are neglected which is a valid approximation for low-level flow velocity as in the considered application. The flow is considered two-dimensional and the velocity field is fully described by two in-plane components. Flow in thickness direction is neglected.

As one can see in the numerical scheme described by Equations (3) to (5), the values for the physical quantities in a considered cell i at the new time step $n+1$ after a time step Δt are calculated from the values of the physical quantities in the considered and neighbouring cells at the old time step n and by geometric properties, for example cell size V_i, interface area $A_{i,j}$ between cells i and j and also the distance vector $\mathbf{l}_{i,j}$ between neighbouring cell centers for the gradient evaluation.

Before calculating the flow velocity between cell V_i and neighbouring cell V_j at the cell boundary, the u- and v-coordinates of the neighbouring cells are rotated into the flattened geometry and transformed into the local coordinate system of cell i. Figure 4a shows cells i and j in the flattened setup and Fig. 4b shows the local coordinate systems in cells i and j.

Initialization. The cell values for the physical quantities are saved in arrays. Consequently, arrays for the physical quantities mass density, x- and y-velocity in the first and second direction of the local cell coordinate system, pressure and

filling fraction are allocated for an old and a new time step. At the beginning of a simulation the arrays for the old time step are filled with the initial condition.

Handling of Boundary Conditions. In contrast to commercial CFD packages, inflow and outflow boundary conditions are not specified at cell boundaries. Additional cells for injection gates and vents must be added as indicated by the cells highlighted in red color in Subfigure e and f of Fig. 2. The pressure in these cells is prescribed and the governing equations are not solved for these cells.

If a cell face (actually a cell edge in the shell mesh) is not shared with another cell, wall boundary conditions are automatically assigned.

2.2 LCMsim Refactoring

The idea of a software tool for LCM filling simulations, which is easy-to-use for the mold engineer and gives maximum flexibility to the scientist, has led to the initial development of RTMsim and its successor LCMsim. During the attempt to use LCMsim for a Reinforcement Learning application [17], the software showed some architectural flaws, that hindered further development. To enable the usage of LCMsim in a broader range of applications and environments, the original source code was refactored to fit the newly identified needs.

In software engineering, the term refactoring describes the activity of restructuring an application's source code without changing its functionality. Refactoring aims to reduce symptoms of suboptimal software design by reverting imperfections that may have accumulated during development. As a cumulative effect, this usually results in better maintainability and extensibility of the code base as well as performance improvements [1].

This section describes why refactoring of LCMsim was needed, which goals were pursued and how the resulting software was structured. Furthermore, a short description of how to use the software module and the graphical user interface are provided. The refactored Julia module, called LCMsim v2, is available on GitHub alongside an extensive documentation [14].

Need for Refactoring. The original implementation of LCMsim showed some deficiencies which made the project hard to understand, maintain and extend, which in consequence impeded further development. Critical limitations are:

1. Former design decisions did not allow for a more general usage of the software, which will be listed below.
2. The overall structure of the software implementation showed little modularization, which caused these decisions to be spread throughout and to affect the whole monolithic architecture.

These limitations led to a refactoring of the whole project, because single problems could not be fixed separately. Some specific aspects that needed to be changed are:

1. One major problem was that the implementation only allowed the user to specify up to five sets of cells for parameterization, which is too low for some of the intended applications. This affected big parts of the preparation routine, since the five possible parameter sets were implemented statically.
2. The behaviour of the simulation in terms of writing to the file system and a graphical user interface was strongly entangled with the functionality, i.e. the numerical calculations.
3. The implementation of different physical models of calculation was distributed through the whole code and realized by conditional statements. Therefore, changing a model or adding a new one would have affected large parts of the code and this is prone to introducing errors.
4. During refactoring, some possibilities for performance optimization were found. The original software performed some disadvantageous loops where the number of iterations could be reduced or computationally expensive operations could be eliminated. The latter were identified by profiling. For example, it was found that the usage of the backslash operator for finding the least-square solution of the over-determined linear system `Amat*xvec=bvec` with a 3×2-matrix `Amat` is less efficient than a multiplication of a transposed matrix by a matrix `Aplus=transpose(Amat)*Amat`, a multiplication of a transposed matrix by a vector `bvec_mod=transpose(Amat)*bvec` and a few algebraic operations for explicitly inverting a 2×2 matrix `Aplus` and subsequent multiplication by a vector `bvec_mod`. Consequently, the code portion

```
xvec=Amat\bvec
dpdx=xvec[1]
dpdy=xvec[2]
```

was replaced by the mathematically equivalent code

```
Aplus=transpose(Amat)*Amat
a=Aplus[1,1]
b=Aplus[1,2]
c=Aplus[2,1]
d=Aplus[2,2]
bvec_mod=transpose(Amat)*bvec
inv = 1/(a * d - b * c)
dpdx = inv * d * bvec_mod[1] - inv * b * bvec_mod[2]
dpdy = -inv * c * bvec_mod[1] + inv * a * bvec_mod[2]
```

Requirements. Five high level requirements, including both functional and non-functional, were identified for the refactoring of LCMsim:

1. A critical non-functional requirement for numerical simulation is efficiency. During mold design, filling simulations can study different manufacturing concepts (i.e. placement of injection gates and vents) to guarantee complete filling. For the engineer it can be tiring to repeatedly wait for results. For machine learning applications it is mandatory to have a fast and efficient simulation to enable model training with realistic computational capabilities.
2. A needed functionality is the full access to the parameterization of each cell. This is important for:

   (a) the free placement of injection gates and vents, i.e. specifying groups of cells as such,
   (b) to turn these on/off, or to change the pressure level at gates and vents, and
   (c) the temporal and spatial variation of the preform properties.
3. Allow for start and stop of the simulation at arbitrary time stamps.
4. The solver and the resulting data need to be efficiently accessible for different user groups. This includes the installation and the integration into other projects such as a machine learning infrastructure [17] or a graphical user interface.
5. A non-functional requirement regarding the source code is extensibility, as the goal is to create a software that fits the technical needs described above, while receiving updates from ongoing research in the fields of composite manufacturing and numerical methods. The simulation shall model different LCM technologies and support different numerical models but the source code should still be clearly readable and extendable by further models without having to change existing code (Open-Closed Principle) [8].

The Julia Programming Language [3] was chosen for this project because:

- Julia is a fast, high-performance language with just-in-time compilation.
- Julia is easy to install and available for various operating systems.
- Julia provides a simple and expressive syntax.
- LCMsim predecessors are available in Julia.

One arguable disadvantage of Julia is that it is not object-oriented and consequently does not allow for some usual and well proven software engineering practices. Due to the relatively small size of this project, the difficulties were maintainable and the advantages overruled.

Architecture. This section gives an overview of LCMsim v2's architecture. Code interna and data structures are explained when needed, but avoided if possible. The source code has been restructured by refactoring into four functional units:

1. Mesh preparation
2. Simulation setup
3. Flow problem solver
4. Collection of convenience and output functions

This modularization applies the Single-Responsibility-Principle, as single parts of the source code fulfill separated tasks and could be changed without affecting others [8]. The graphical user interface has been moved to an external package that accesses the public interface of the LCMsim v2 package.

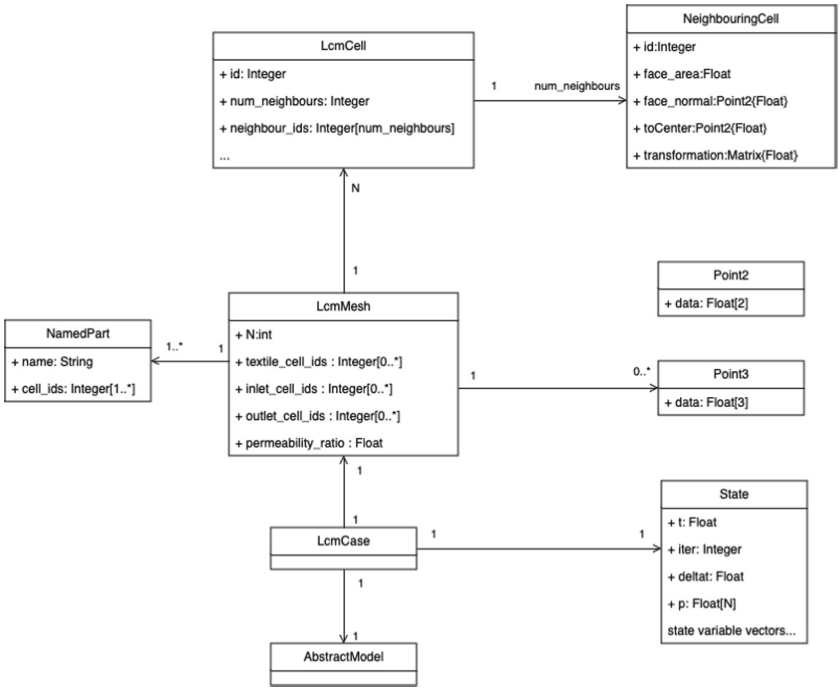


Fig. 5. The central data structures of LCMsim v2. Properties irrelevant for the overall understanding were omitted for readability. Julia structs are strictly seen different from classes, however a UML class diagram could be used here to model their relations.

Mesh Preparation: Reading different mesh formats is possible. Based on the file extension, the corresponding parsing routine is selected and the mesh is translated to a unified program representation.

The raw mesh data which just contains the positional, connectivity and grouping information of the mesh cells is enriched with user provided part information. Context and material parameters specified for sets are assigned to the cells.

Many geometric properties referring either to the individual cells or their connections are needed to solve the governing equations, see Sect. 2.1. These are calculated a priori to avoid their highly repeated computation. Opposing to the original implementation, the mesh preparation is done by a collection of subroutines which are called by a main routine. This makes this step easier to maintain and extend.

As Julia structs[2] are immutable[3] by default, the information is transferred to a structured representation of the mesh (i.e. `LcmMesh`) only at the very end of the mesh preparation. The data structures can be seen in Fig. 5. The main component of the `LcmMesh` is an array or, in Julia terms, a vector of `LcmCell`s. Each `LcmCell` represents a triangular element and carries information about itself as well as information about its neighbours. The self-referring properties consist of the beforementioned material properties and geometric parameters (for example the cell's permeability or area). Those are left out in the `LcmCell` block for brevity. Additionally, each cell carries information about each of its neighbours, which are grouped in the `NeighbouringCell` struct. This allows to efficiently access these values when iterating over the cell vector to solve the governing equations. This can be seen as a trade-off between memory consumption and runtime.

Consider the following example. The pressure gradient must be evaluated at every time step for every cell. For this calculation the vectors between considered and neighbouring cell centers are required. If these vectors were not precalculated and accessible through the considered cell's struct, this would require multiple look-ups to retrieve the neighbouring cells and their centers. This pattern applies to many of the properties which are saved in `LcmCell` and `NeighbouringCell`.

As runtime efficiency was identified as a critical requirement, the decision easily fell towards higher memory consumption and reduced computational complexity. The same procedure was chosen for the index shortcut fields of `LcmMesh` (namely `textile_cell_ids`, `inlet_cell_ids` and `outlet_cell_ids`). These allow for efficient access to all cells of these types, without any extra checks and look-ups, but again at the cost of memory.

Simulation Setup: This step mainly consists of parsing the user-provided simulation parameter file and creating the initial state. The `State` struct describes the time varying variables of a simulation case. The vectorial fields can be matched to the cells of the mesh that this state has been created from.

A key concept of LCMsim v2 is the incorporation of multiple physical models. An argument to the simulation preparation routine indicates the choice.

The physical models are programmatically represented by structs, which carry model-specific physical parameters and whose type indicates the chosen physical model.

The model types are related to each other by inheritance. Similarly to the mesh creation, a struct of the chosen model's type is instantiated at the end of the preparation routine. The models, their relations and their properties are visualized in Fig. 6.

The initial state is then created using the newly created model's parameters.

[2] A struct in Julia is a composite data type that allows to store multiple values in a single object.

[3] Mutable structs are possible, but their usage is usually discouraged for efficiency reasons.

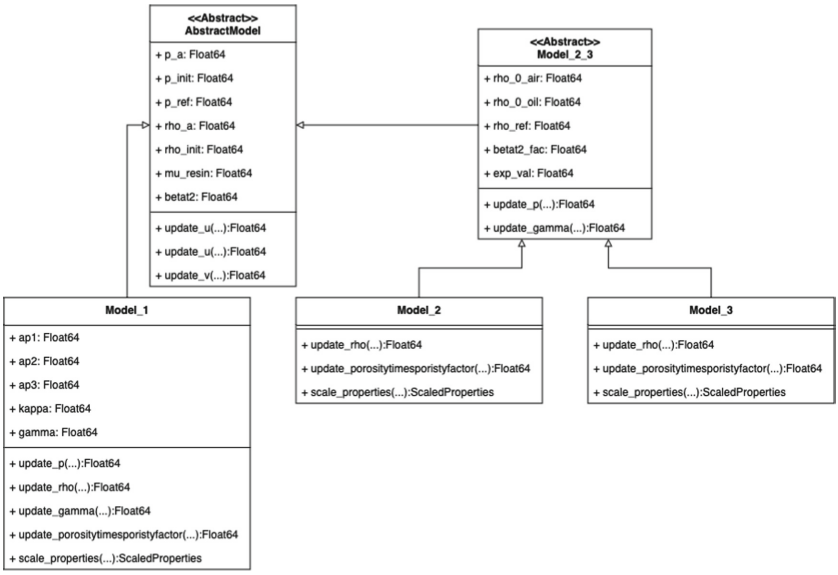


Fig. 6. Inheritance structure representing the physical models. Even though Julia not being object oriented, the struct types were modeled as classes where relevant functions mainly associated with a type were interpreted as methods of the respective class. Note `ScaledProperties`, which is a simple data structure that is created temporarily for each cell and used by the state update functions. It allows to apply a scaling to the parameters it contains, which can then retrieved by the update functions in a unified manner.

Flow Problem Solver: The core functionality of LCMsim v2 is contained in one function which takes a model instance, a mesh and a state as input. It solves the IBV problem for the chosen physical model up to a specified end time. It consists of a main time-evolution loop and the performed steps are typical for the numerical solution of an IBV problem.

An abstracted version of the main loop is depicted in Algorithm 1. It incorporates the different physical models in one clearly readable code snippet by using the programming pattern of template methods.

As introduced earlier, each physical model is represented by a subtype of `AbstractModel`. In the main loop, an update function for each state variable is called with an instance of the physical model. By supplying multiple implementations for each function signature, the developer can realize the different physical models with little effort. The implementation to use is chosen at runtime with respect to the given model type.

Algorithm 1. Abstract depiction of the solve function

Require: $mesh : LcmMesh, state_i : State, t_{next} : Float64, model : AbstractModel$
  $t_i \leftarrow state_i.t$
  $\Delta t \leftarrow state_i.\Delta t$
  $state_{i+1} \leftarrow state_i$
  while $t_i \leq t_{next}$ **do**
    for $cell \in mesh.textile\_cells$ **do**
      - Scale parameters
      $scaled\_properties \leftarrow scale\_properties(model, ...)$
      - Calculate the pressure gradient
      - Evaluate the numerical flux functions, i.e. sum term in Eqs. 3-5
      - Calculate new values for state variables according to 3-5:
      $state_{i+1}.rho[cell] \leftarrow update\_rho(model, scaled\_properties, \Delta t, ...)$
      ...
      - Overwrite old state of this cell
      $state_i.rho[cell] \leftarrow state_{i+1}.rho[cell]$
      ...
    end for
    - Calculate next time step
    $\Delta t \leftarrow adaptive\_timestep(model, \Delta t, ...)$
    $t_i \leftarrow t_i + \Delta t$
  end while
  - Return updated state
  $state_{i+1}.t \leftarrow t_i$
  $state_{i+1}.\Delta t \leftarrow \Delta t$ **return** $state_{i+1}$

In Algorithm 1, this mechanism is applied for example at `scale_properties()` and `update_rho()`, which behave differently according to the model type. For this to work, all implementations of one specific update formula need to share the exact same signature, i.e. the same argument and return types. They may only differ in the type of the model, which then determines which implementation is effectively used. Since it is not possible in Julia to force the implementations of certain functions, some discipline is required to make use of and maintain this structure. This is a shortcoming of Julia compared to most object-oriented languages where concepts such as abstract classes with abstract methods can be employed to force the implementation of these methods by specific subclasses already at compile time.

As some models share certain update formulas, it is possible to build a tree-like inheritance structure with `AbstractModel` as the root, where the models sharing these calculations are summarised under an abstract parent node. The Julia compiler chooses the implementation for the most specific type if multiple versions of a function are available. Thus the developer has to provide all implementations for a state variable update that are as specific as necessary but as general as possible according to the inheritance tree. In order to avoid confusion only one implementation should be provided per branch of the inheritance tree.

The currently implemented models can be seen in Fig. 6. This structure is compliant to the Open-Closed-Principle in a way that new numerical models can be added without modifications to the main simulation loop. Another benefit is a quick overview of the similarities and differences of the numerical models.

The class diagram in Fig. 6 lists the specific implementations of model functions. Note that every model of `Model1 - Model3` has the same set of functions with implementations given either in the type itself or in a super type. Sometimes this leads to trivial implementations which we accept in favor of the simplicity and readability of the code.

For example, `scale_properties()` is only used in `Model3` and for `Model1` and `Model2` the function does not actually scale the values.

Convenience Functions: The end user is not meant to access directly the solve function described before. Therefore, different convenience functions are provided which serve as public interface to prepare and retrieve data and to invoke the solver. The mesh, the part description and the simulation parameters always need to be provided as files respectively their paths. The results of the preparation routine can be directly saved to a given path or retrieved as runtime objects. Depending on the application, it can be more handy to operate only on saved data or to keep everything in the memory. Both variants can be used to start or continue solving.

The datatype used for this high level communication is `LcmCase`, as can be seen in Fig. 5.

When saving results to the disk, two data formats are available: JLD2 and HDF. JLD2 is a Julia package which allows serialization of Julia objects. When using this option, a `LcmCase` instance is saved with the latest state being included. The other variant is to save data as a HDF file. HDF is data format used to efficiently store and load hierarchically organized data. When using this option, the resulting file contains sufficient information to recreate the mesh and the simulation parameters. It is also possible to save a series of states, which is for example useful to create animations of the flow front propagation. A complete documentation of the file structure of the HDF output can be found at the LCMsim v2 repository [14]. Both formats can be used to load and continue previously saved simulations.

Running a Simulation. This section shortly summarizes the necessary steps to perform a simulation with LCMsim v2.

LCMsim v2 does not include mesh generation as meshing is considered an external task[4]. A three-node triangular shell mesh must be imported. Depending on the mesh format, cell sets which are assigned with different preform properties have to be specified differently. Currently, three different mesh formats are implemented and the selection is based on the file extension:

[4] Mesh generation is possible with commercial software packages (for example Altair HyperWorks, Ansys or Abaqus) and free software packages (for example Salome-Meca, PrePoMax, Gmsh or Netgen).

Table 1. Additional input for the parts defined by the sets in the mesh file.

Column name	Description	Value range
`name`	Name of the part. Will be used later on to match action to inlets/outlets	Freely choosable (reasonable)
`type`	Type of this part	{base, inlet, outlet, patch}
`part_id`	The physical part ID assigned to the part in the mesh file.	Determined by mesh file.
`thickness`	Cavity thickness.	$> 0.$
`permeability`	Permeability in first principal direction.	$> 0.$
`permeability_noise`	Standard deviation applied to permeability values with a normal distribution.	0-0.5 (no definitive cap but unrealistic values break simulation).
`porosity`	Porosity.	$\in (0., 1.)$
`porosity_noise`	Standard deviation applied to porosity values with a normal distribution.	0-0.5 (no definitive cap but unrealistic values break simulation).
`porosity_1`	Porosity at pressure `p_1`.	$\in (0., 1.)$
`p_1`	Pressure corresponding to porosity `p_1`	$> 0.$
`alpha`	Factor for permeability in second principal direction.	Real
`refdir_1`	x-,	Real
`refdir_2`	y-,	Real
`refdir_3`	z-component of reference direction which will be projected onto the cells to give the first principal direction.	Real

- `.dat`: Nastran format with nodes, elements and element sets.
- `.bdf`: Nastran format with nodes, elements and property ID assigned to elements.
- `.inp`: Abaqus format with nodes, elements and element sets.

The parts defined by the sets in the mesh file need additional input parameters for the simulation. These are provided via a `.csv`-file with the content shown in Table 1. For example, the following snippet describes the properties for the validation case 1 from [13]:

```
name,type,part_id,thickness,porosity,porosity_noise,permeability,
    permeability_noise,alpha,refdir1,refdir2,refdir3,
    porosity_1,p_1
base,base,1,3e-3,0.7,0.0,3e-10,0.0,1,1,0,0,0.7,0.1e5
central_inlet,inlet,2,3e-3,0.7,0.0,3e-10,0.0,1,1,0,0,0.7,0.1e5
```

The physical parameters for the simulation need to be provided by a `.csv`-file with the content shown in Table 2. For example, the following snippet describes the parameters for the validation case 1 from [13]:

```
p_ref,rho_ref,gamma,mu_resin,p_a,p_init,rho_0_air,rho_0_oil
1.01325e5,1.225,1.4,0.06,1.35e5,1.0e5,1.2,960
```

With these three files prepared, the solver can be started using one of the functions from the public interface.

GUI. The graphical user interface is provided via a separate repository [10]. The GUI solely depends on functions from LCMsim v2's public interface, such that the functional internals of both modules are fully decoupled.

The GUI was developed for Windows operating systems and is launched via a batch file which starts Julia with a configuration file as argument, i.e. `julia -L lcmsim_config.jl`. The configuration file specifies:

- if LCMsim is started in batch mode where files for the part properties, simulations parameters and mesh are used or if parameters are filled in the mask,
- which model type is started (2: model from [15], 4: model 2 with degree-of-cure equation),
- which mesh type is used (1: `.dat`, 2: `.inp`, 3: `.bdf`),
- the path to the `LCMsim_v2.jl` repository,
- the path to the GUI repository.

The following configuration file is used to start the GUI shown in Fig. 7:

```
i_batch=1
i_model=2
i_mesh=1
mypath=joinpath(pwd())
repositorypath="D:\\work\\github\\LCMsim_v2.jl"
guipath="D:\\work\\github\\LCMsim_GUI\\gui_and_cases\\gui"
include(joinpath(guipath,"lcmsim_v2_gui_gtk4.jl"))
```

Clicking the button `Run with input files` calls the `create_and_solve` function from the public interface.

Table 2. Physical parameters for the simulation.

Column name	Description
`p_ref`	Reference pressure (only for model type 1)
`rho_ref`	Reference mass density (only for model type 1)
`gamma`	Adiabatic index (only for model type 1)
`mu_resin`	Dynamic viscosity of the oil/resin
`p_a`	Pressure at the inlets
`p_init`	Initial cavity pressure
`rho_0_air`	Density of air at `p_init`
`rho_0_oil`	density of oil/resin at `p_a`

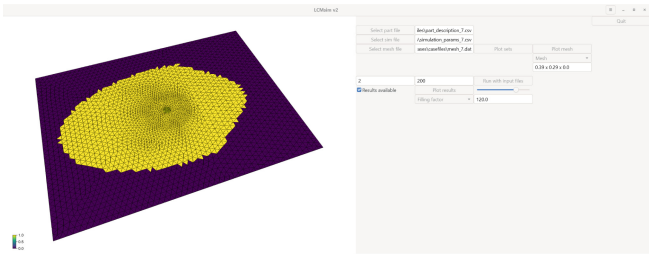


Fig. 7. GUI for starting a flow simulation run with files for part properties, simulation parameters and mesh.

3 Results and Discussion

The improvements from the refactoring are demonstrated by a comparison of the computational time for test case 1 of [15] (radial flow experiment in a permeameter with central inlet and two preform regions with different in-plane permeability) and by adding an additional conservation law for the degree of cure.

In Sect. 3.1, a first comparison verifies that the simulation results have not changed due to the refactoring step. In Sect. 3.2, a second study shows which code parts have to be adapted in order to integrate the effect of viscosity on mold filling when involving fast curing thermosets.

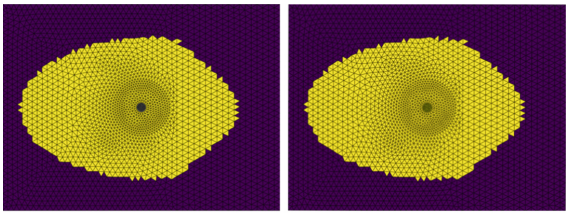
3.1 Performance Comparison

The relevant geometric parameters are as follows (test case 1 from [15]): Flat plate cavity ($390\times 290\times 3.14$ mm^3) with central injection port (13 mm diameter). The main preform covers the leftmost preform portion (245 mm wide) and the manipulated preform with lower permeability and porosity covers the rightmost section (145 mm wide). Initial cavity pressure is ambient pressure 100000 Pa and injection pressure is 191000 Pa. The fluid properties are mass density 960 kg/m^3 and dynamic viscosity 0.071 Pas. Different levels of flow front propagation velocity are expected in the different preform zones.

The flow experiment is simulated with both tools, i.e. LCMsim and LCMsim v2, respectively, on a hardware which is state-of-the-art for mechanical engineering simulations (Processor Intel Core i7-12800H, 64 GB RAM). The simulation for the whole 160 s filling time takes 490 s with LCMsim v2, showing an improvement by a factor of approximately 17 over LCMsim. A similar factor is achieved for other cases since the improvement comes from the time per iteration (i.e. for one time step) and the adaptive time stepping was not changed.

The filling after 120 s simulated with LCMsim and LCMsim v2 is shown in Fig. 8 and shows negligible differences. The difference is measured by comparing the filled areas. The two filing contour plots from Fig. 8 are converted into binary pictures of equal size where the black area covers the filled area. Then, the

numbers of black mesh cells are used to calculate a relative error in the filled area. The filled area is nearly identical (< 0.5 % difference). This difference in the filled areas is illustrated in Fig. 9 where transparent filling patterns are superimposed: Only five cells (in light grey) at the top show a difference in filling.



(a) (b)

Fig. 8. Filling for test case 1 after 120 s: (a) LCMsim and (b) LCMsim v2.

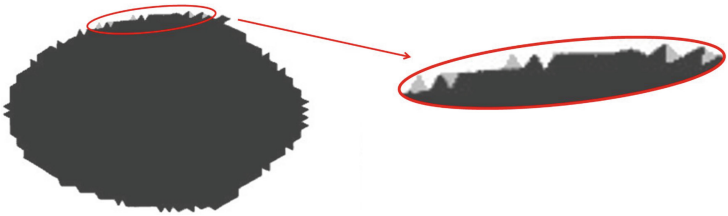


Fig. 9. Superposition of filled areas simulated with LCMsim and LCMsim v2, respectively, for test case 1 after 120 s. Light grey cells represent cells with different filling fraction.

3.2 Extending the Scope of Simulation

If a fast curing thermoset is used instead of oil as test fluid, curing of the resin must be considered during mold filling. The resin will undergo a chemical change due to the cross-linking reaction and will then form a highly cross-linked solid.

The degree-of-cure α is a variable used to describe the state of cure of the resin. It ranges from 0 to 1. When the degree of cure is 0, the resin is uncured, and when the degree of cure is 1, the resin is completely cured.

Depending on the resin system used and the type of chemical reactions there are several kinetic models available in literature for the cure reaction of thermosetting resins [5]. The most widely used and the simplest is the n^{th}-order reaction model.

The degree-of-cure equation with convection which has to be implemented in LCMsim v2 is:

$$(\varepsilon\gamma\alpha)_t + \mathbf{u} \cdot (\gamma\alpha) = S_\alpha \tag{6}$$

with porosity ε, filling fraction γ, superficial velocity $\mathbf{u}$, source function $S_\alpha = k_{\text{doc}}(1-\alpha)^{n_{\text{doc}}}$ and modeling parameters k_{doc} and n_{doc}. Similar to the implementation of the filling fraction in RTMsim [13] the equation is written in conservative form as:

$$(\varepsilon\gamma\alpha)_t + \nabla \cdot (\gamma\alpha\mathbf{u}) - \gamma\alpha\,(\nabla \cdot \mathbf{u}) = S_\alpha \tag{7}$$

The viscosity change with the degree of cure is described by:

$$\mu = \mu_0 e^{k_\mu \alpha} \tag{8}$$

with uncured dynamic viscosity μ_0 and modeling parameter k_μ.

In total, three additional parameters for the modeling equations (k_{doc}, n_{doc} and k_μ) and two parameters for the initial and boundary conditions ($\alpha_{\text{init}} = 0$ and $\alpha_{\text{a}} = 0$) are required.

A branch `doc_eq` was created on the GitHub repository where all the changes, described in Appendix A were implemented. The demonstrated procedure can be generalized for modifying existing or adding new conservation laws, for example a temperature equation.

Results for Filling Simulation with Degree-of-Cure. The considered test case is the same as in the previous section. Figure 10 shows the flow front after 200 s with (i) constant viscosity and (ii) degree-of-cure dependent viscosity. Parameters used are $k_{\text{doc}} = 0.007$, $n_{\text{doc}} = 1.2$ and $k_\mu = 0.0$ and 1.5 respectively, for case (i) and (ii). The parameters are chosen such that $\gamma > 0.5$ after 120 s and the flow front slows down significantly.

As expected, the filling for (i) is equal to the filling from the previous section. One can see numerical diffusion in the degree-of-cure at the flow front. This could be avoided by plotting the product of the binary filling fraction times degree-of-cure instead of the degree-of-cure. By plotting the product one gets a non-zero degree-of-cure only if already filled which is reasonable. The filling with degree-of-cure dependent viscosity shows a slower flow front propagation as expected.

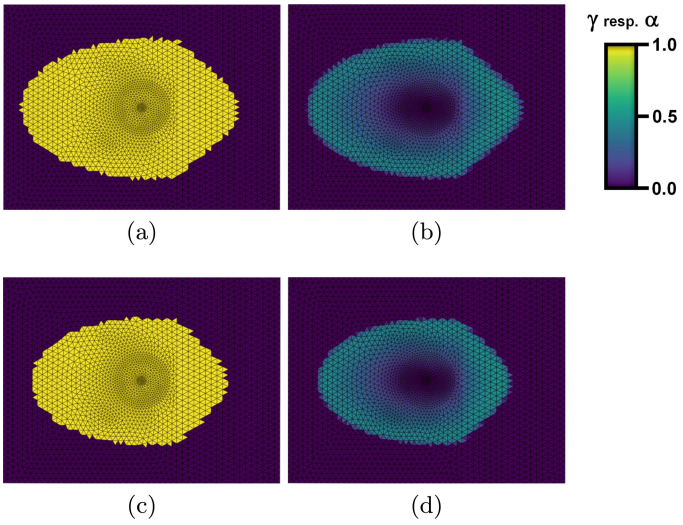


Fig. 10. Filling fraction γ (left) and degree-of-cure α (right) with constant (top) and variable (bottom) viscosity. The increase of the degree-of-cure caused by the chemical cross-linking reaction causes an increase in fluid viscosity and thus, a deceleration of flow propagation.

4 Conclusion and Outlook

The work at hand describes code modifications when a software engineer meets self-fabricated engineering software. The Julia package LCMsim for filling simulations in liquid composite molding was analyzed and requirements for a refactoring step were formulated. After refactoring, the required computational time for a typical benchmark problem is reduced by a factor of 17. Furthermore, adding supplementary equations describing phenomenological mechanisms was facilitated. The refactored version is available on a Github repository [14] where in a separate branch all described changes for adding a supplementary equation are shown in detail.

As a next step, new equations (for example the temperature equation) and new functionalities (for example for vacuum-assisted resin infusion) will be implemented. Improved performance together with the opportunity to customize the modelled physical and chemical processes constitute a very useful tool for engineers and scientists in the field of composite manufacturing.

A Code Preparation

The following code modifications are necessary to solve an additional equation:

– The state vector will be extended for the variable `doc` (degree of cure).

- A new model type with ID 4 is created. The structure `Model_4` extends `Model_2_3`. All structures have additional parameters used to describe the changes in the degree of cure and the viscosity (`k_doc`, `n_doc`, `k_mu`, `doc_init`, `doc_a`). Otherwise the states cannot be initialized.
- The `csv`-file with the simulation parameters optionally (in order to be backward compatible with the already existing model types) can have three additional parameters (`k_doc`, `n_doc`, `k_mu`; the values for `doc_init=0`, `doc_a=0` need not be read).
- The functions `save_model` and `load_model` must be extended for model type 4 and for the five additional parameters in all model types.
- The function `save_state` must be extended for the variable `doc`.
- The function `create_initial_state` must be extended for the variable `doc`.
- In file `abstract_model.jl`, create a dummy function `update_doc` with returned value 0. and function `update_viscosity` must have an additional input argument `doc_new`.
- The file `model_4.jl` is created as copy of `model_2.jl`. Appropriate functions for the degree-of-cure `update_doc` and the viscosity update `doc_new` must be created.
- Include `model_4.jl` in file `LCMsim_v2.jl`.
- In function `solve`:
  - Create a vector for the old state `doc_old` and deep copy the values of the values from the vector with the old into the vector with the new state `doc_new`.
  - Inside the time evolution loop, call the functions for the numerical flux calculation and the degree-of-cure update, add the additional argument to the viscosity update function and assign the new values to the old values of the degree-of-cure at the end of one time step.
  - Add the degree of cure to the return of the final state.
- The numerical scheme itself is implemented in the functions `update_doc` and `aggregate_neighbour_flux_doc` which calls the functions `numerical_flux_function_doc` and `numerical_flux_function_boundary_doc`. The functions with the appendix `_doc` are modified copies of the corresponding functions without this appendix.

References

1. Refactoring: Improving the Design of Existing Code. Addison-Wesley Longman Publishing Co., Inc., USA (1999)
2. Barandun, G., et al.: Development of a next generation composite - annulus filler for rolls royce jet engines. In: SEICO 13 SAMPE EUROPE 34th International Conference and Forum (2013)
3. Bezanson, J., Edelman, A., Karpinski, S., Shah, V.B.: Julia: a fresh approach to numerical computing. SIAM Rev. **59**(1), 65–98 (2017)
4. Courant, R., Friedrichs, K., Lewy, H.: Über die partiellen Differenzengleichungen der mathematischen Physik. Math. Ann. **100**, 32–74 (1928)

5. Dodiuk, H., Goodman, S.: Handbook of Thermoset Plastics, vol. 254, January 2014
6. ESI Group: PAM-COMPOSITES: Composites Simulation Software, https://www.esi-group.com/products/pam-composites
7. Ferziger, J.H., Perić, M.: Computational methods for fluid dynamics. Springer (2002)
8. Martin, R.: Agile Software Development: Principles, Patterns, and Practices. Alan Apt series, Pearson Education (2003)
9. Niu, C., Niu, M.: Composite airframe structures: practical design information and data. Adaso Adastra Engineering Center (1992)
10. Obertscheider, C.: GUI for refactured LCMsim (v2). https://github.com/LCMsim/LCMsim_GUI (2024)
11. Obertscheider, C., Fauster, E.: RTMsim - a Julia module for filling simulations in resin transfer moulding. https://github.com/obertscheiderfhwn/RTMsim (2022)
12. Obertscheider, C., Fauster, E.: LCMsim - a Julia module for filling simulations in Liquid Composite Moulding. https://github.com/obertscheiderfhwn/LCMsim (2023)
13. Obertscheider, C., Fauster, E.: RTMsim - a Julia module for filling simulations in Resin transfer Moulding with the finite area method. J. Open Source Soft. **8**(84), 4763 (2023)
14. Obertscheider, C., Fauster, E., Heber, L.: LCMsim_v2.jl - Julia module for the refactured LCMsim (v2). https://github.com/LCMsim/LCMsim_v2.jl (2024)
15. Obertscheider, C., Fauster, E., Stieber, S.: Experimental validation of a new adaptable LCM mold filling software. Adv. Manuf. Polymer Compos. Sci. **9**(1), 2282310 (2023)
16. Reddy, J.N.: An Introduction to Continuum Mechanics. Cambridge University Press, 2 edn. (2013)
17. Stieber, S., Heber, L., Obertscheider, C., Reif, W.: Control of composite manufacturing processes through deep reinforcement learning. In: 2023 International Conference on Machine Learning and Applications (ICMLA), pp. 17–22 (2023)
18. Stieber, S., et al.: Inferring material properties from FRP processes via sim-to-real learning. Int. J. Adv. Manuf. Technol. **128**, 1–17 (2023)
19. Strong, A.: Fundamentals of Composites Manufacturing, Second Edition: Materials. Society of Manufacturing Engineers, Methods and Applications (2008)
20. Versteeg, H., Malalasekera, W.: An Introduction to Computational Fluid Dynamics: The Finite Volume Method. Pearson Education Limited (2007)